Análise de padrões de projeto — Açaí do Dev

Nome: João Comini Cesar de Andrade

Foram identificados 6 padrões do catálogo refactoring.guru: 2 criacionais (Singleton, Factory Method), 2 estruturais (Decorator, Facade) e 2 comportamentais (Observer, Strategy). Os números de linha do console citados na tabela são do console real da **sessão de uso** descrita abaixo (a numeração começa em 01 a cada carregamento da página).

Sessão de uso registrada (origem das linhas citadas)

(a) Carregamento da página: linhas 01–04. **(b)** Cliques com o carrinho ainda vazio: *Finalizar pedido* (05), *Reabrir carrinho* (06), *Tentar new Carrinho()* (07), *Reabrir carrinho* (08) e *Finalizar pedido* (09). **(c)** Montagem de 500 ml + granola + banana + nutella, frete Sedex, aviso por SMS e clique em *Adicionar ao carrinho*: linhas 10–18 (a linha 16 é o cálculo de frete do resumo). **(d)** Clique em *Finalizar pedido*: linhas 18–27 (a linha 18 é o cálculo do frete feito logo no início de **finalizar()**).

Padrão	Categoria	Onde está (classes / métodos)	Justificativa (estrutura no código)	Evidência no console (nº das linhas)
Singleton	Criacional	Classe **Carrinho**: campo estático privado **#instancia**, **constructor** e método estático **obterInstancia()**. Uso: **carrinhoDaLoja** e botões "reabrir" e "new Carrinho()".	A própria classe controla sua instanciação. **obterInstancia()** só chama **new Carrinho()** se **#instancia** ainda for nulo; caso contrário devolve a referência já guardada. O **constructor** verifica **#instancia** e lança Error se já houver um carrinho, impedindo que qualquer código de fora crie um segundo. Quem precisa do carrinho (a tela) não o cria: pede à classe, que garante uma única instância e um ponto global de acesso — a definição do catálogo (Singleton: garantir que uma classe tenha apenas uma instância e fornecer um ponto de acesso global a ela). O contador estático **#contador** existe apenas para tornar isso visível no console.	**01**: obterInstancia() "ainda não existe nenhum: criando o carrinho nº 1". **06, 08**: "já existe um aberto: devolvo sempre o mesmo (nº 1)". **07**: new Carrinho() bloqueado com erro "Já existe um carrinho aberto. Use Carrinho.obterInstancia()".
Factory Method	Criacional	Criador: **CanalDeAvisos** (**criarAviso()** abstrato + **avisar()**). Criadores concretos: **CanalDeAvisosEmail / SMS / Push**. Produto: **Aviso**; produtos concretos: **AvisoEmail / SMS / Push**. Cliente: **avisarCliente()**.	**avisar()** (na classe base) contém a lógica comum e chama **this.criarAviso()** sem saber qual classe concreta será instanciada; o método base apenas lança erro. Cada subclasse sobrescreve **criarAviso()** e decide qual produto criar (**new AvisoSMS()**, **new AvisoEmail()**, **new AvisoPush()**). Ou seja, a criação do objeto fica num método que as subclasses redefinem, e o código que usa o aviso trabalha só com a abstração Aviso (enviar). É a definição de Factory Method: uma interface para criar objetos que deixa as subclasses decidirem qual classe instanciar. **avisarCliente()** apenas escolhe qual criador usar a partir do canal selecionado.	**23**: "CanalDeAvisosSMS.criarAviso() produziu um AvisoSMS" (o criador concreto decidiu o produto). **24**: o produto criado envia o SMS. Com outro canal, a linha 23 mostraria AvisoEmail ou AvisoPush, sem mudança em **avisar()**.
Decorator	Estrutural	Componente concreto: **Acai**. Decorador base: **Adicional** (guarda **deBaixo**). Decoradores concretos: **Granola, Banana, Nutella, LeiteEmPo**. Montagem: **montarAcai()**.	**Acai** e **Adicional** expõem a mesma interface (**descricao()** e **custo()**). **Adicional** recebe no construtor outro objeto com essa interface (**deBaixo**) e o *envolve*: seus métodos delegam para **deBaixo** e acrescentam a sua parte (rótulo e preço). Em **montarAcai()** o laço **item = new Classe(item)** embrulha o objeto anterior a cada adicional escolhido, formando uma cadeia em qualquer combinação, sem precisar de uma subclasse para cada mistura (AcaiComGranolaEBanana etc.). É o Decorator do catálogo: anexar novas responsabilidades a um objeto dinamicamente, colocando-o dentro de invólucros que as implementam.	**10, 11, 12, 13**: cada linha é uma camada nova; a descrição cresce (Açaí 500 ml, + granola, + banana, + nutella) e o custo se acumula (R$ 12,90 -> 15,90 -> 18,40 -> 22,40). **14**: o objeto final guardado responde com a cadeia inteira. **21**: o pagamento cobra R$ 22,40, o **custo()** da cadeia completa.
Facade	Estrutural	Fachada: **Checkout.finalizar()**. Subsistemas: **Estoque.reservar()**, **Pagamentos.processar()**, **Entregas.agendar()** e o envio do aviso (**avisarCliente()**).	O **constructor** de **Checkout** cria os três setores e **finalizar()** conduz a sequência correta (reservar, cobrar, agendar, avisar, esvaziar o carrinho). O botão de finalizar faz uma única chamada, **new Checkout().finalizar(...)**, e não conhece **Estoque**, **Pagamentos** nem **Entregas**, que por sua vez não sabem da existência da fachada. Uma interface simples esconde várias classes com interfaces diferentes — a definição de Facade: interface simplificada para um conjunto complexo de classes.	**19**: Checkout declara "eu resolvo estoque, pagamento, entrega e aviso num método só". **20** (estoque), **21** (pagamentos), **22** (entregas), **23–24** (aviso) e **25** (conclusão): vários subsistemas acionados por um único clique em Finalizar pedido.

Padrão	Categoria	Onde está (classes / métodos)	Justificativa (estrutura no código)	Evidência no console (nº das linhas)
Observer	Comportamental	Canal de eventos: **Barramento** (**assinar()** = inscrever, **publicar()** = notificar). Publicadores: **Carrinho.adicionar()** ("item-adicionado") e **Checkout.finalizar()** ("pedido-finalizado"). Assinantes: três funções registradas na "fiação da tela".	Os assinantes se inscrevem em tempo de execução com **barramento.assinar(evento, função)**, que os guarda numa lista por evento. Quando algo acontece, o publicador só chama **publicar(evento, dados)**, e o **Barramento** percorre a lista chamando cada assinante. O **Carrinho** não conhece o selo, a promoção nem a tela: não há dependência direta. Cada assinante reage à sua maneira (atualizar selo; ativar promoção só se totalItens >= 2; refazer a tela). É o Observer do catálogo: mecanismo de assinatura para notificar vários objetos sobre eventos de outro (em variante publicar/assinar, com canal intermediário).	**02, 03, 04**: inscrições (2 interessados em item-adicionado, 1 em pedido-finalizado). **15**: publicar('item-adicionado') "avisando 2 interessados". **17**: selo atualizado "sem que o carrinho saiba quem o observa" (o 2º assinante foi chamado mas não agiu: só ativa a promoção com 2 itens ou mais, e havia 1). **26, 27**: publicar('pedido-finalizado') notifica e a tela é atualizada.
Strategy	Comportamental	Estratégias: **FretePAC, FreteSedex, FreteRetirada** (mesma interface **rotulo()** / **calcular(pesoKg)**). Seleção: **MODALIDADES_DE_FRETE** e **freteEscolhido()**. Contextos que usam a estratégia: **atualizarResumo()** e **Checkout.finalizar()**.	Três algoritmos de cálculo de frete (fórmulas diferentes; a retirada devolve 0) estão em classes separadas e intercambiáveis, todas com o mesmo método **calcular()**. **freteEscolhido()** instancia a classe correspondente à opção marcada na tela; **atualizarResumo()** e **Checkout.finalizar()** recebem esse objeto e apenas chamam **frete.calcular()**, sem if/switch por tipo de frete. Trocar a modalidade muda o comportamento em tempo de execução sem alterar quem a usa — definição de Strategy: definir uma família de algoritmos, colocar cada um em uma classe e torná-los intercambiáveis.	**16** e **18**: "FreteSedex · R$ 18,90 + R$ 4,00/kg", chamado por **atualizarResumo()** (16) e por **Checkout.finalizar()** (18) pelo mesmo método **calcular()**, sem if/switch. Escolhendo PAC (11,90 + 2,50/kg) ou Retirada ("busca na loja, sem custo"), o mesmo ponto de chamada mostra outra classe.

Objetos que participam de mais de um padrão (papéis diferentes)

Carrinho: no **Singleton** é a classe que se auto-limita a uma instância (linhas 01, 06, 07, 08); no **Observer** é o *publicador* do evento "item-adicionado", pois **adicionar()** chama **barramento.publicar()** (linhas 14–15).

Checkout: na **Facade** é a fachada que orquestra os subsistemas (linhas 19–25); na **Strategy** é um dos contextos que recebe a estratégia de frete e chama **calcular()** (linha 18); no **Observer** é publicador de "pedido-finalizado" (linha 26); e usa o **Factory Method** apenas como cliente, via **avisarCliente()** (linhas 23–24), sem fazer parte da estrutura do padrão.

Item do carrinho (resultado de **montarAcai()**): é a cadeia do **Decorator**; o **Carrinho** e o **Checkout** só o consomem pela interface **descricao()** / **custo()**, sem saber quantas camadas existem.